

宁波大学
NINGBO UNIVERSITY

实 验 报 告

[实验 2 ADD 实验与伪指令实验]

姓名：	赵瑞杰
学号：	216001944
课程名称：	嵌入式系统
所在学院：	信息学院
专业班级：	计算机科学与技术 1 班
任课教师：	王翀 王艺睿

实验项目名称	ADD 实验与伪指令实验

一、实验目的与要求：

- 掌握 ARM 汇编语言的基本使用；
- 熟悉 eclipse 开发工具建立汇编工程和仿真；
- 掌握 ARM 汇编语言的基本使用和一些伪指令的使用；
- 熟悉 eclipse 开发工具建立汇编工程和仿真；

二、实验环境：

Eclipse for ARM、FS_JTAG ARM 仿真器、华清远见 FF4412 开发板

三、实验方法和原理

常用的 RAM 汇编语言指令包括 mov、add、ldr 等。这些指令用于在寄存器之间移动数据、进行算术运算以及访问内存中的数据。

（1）mov 指令：

mov 指令用于将立即数或寄存器中的值移动到另一个寄存器中。

语法：mov destination, source

示例：mov r0, #9 将立即数 9 存入 R0 寄存器。

（2）add 指令：

add 指令用于将两个寄存器中的值相加，并将结果存入目标寄存器中。

语法：add destination, operand1, operand2

示例：add r1, r1, r0 将 R1 和 R0 的值相加，并将结果存入 R1 寄存器。

（3）ldr 指令：

ldr 指令用于加载内存中的数据到寄存器中。

语法：ldr destination, [address]

示例：ldr r1, [r0] 将 r0 寄存器中的地址指向的内存中的数据加载到 r1 寄存器中。

（4）cmp 指令：

cmp 指令用于比较两个操作数的值，并根据比较结果更新标志寄存器的值。通常与条件跳转指令配合使用，用于实现条件分支。

语法：cmp operand1, operand2

示例：cmp r1, #0 比较 r1 寄存器的值与立即数 0。

（5）bne 指令：

bne 指令用于根据比较结果执行跳转操作，当比较结果不等于零时跳转到指定的标签处。

语法：bne label

示例：bne loop 当比较结果不等于零时跳转到标签 loop 处。

四、实验过程、步骤及内容

（1）ADD 实验

实验代码：

ASM Code

```
.globl  _start
_start:
     mov r0,#9 //9 存入 r0
    mov r1,#15    //15 存入 r1
    add r1,r1,r0//r1 与 r0 相加存入 r1
stop:
    b stop

```

实验步骤：

1、导入光盘实验源码

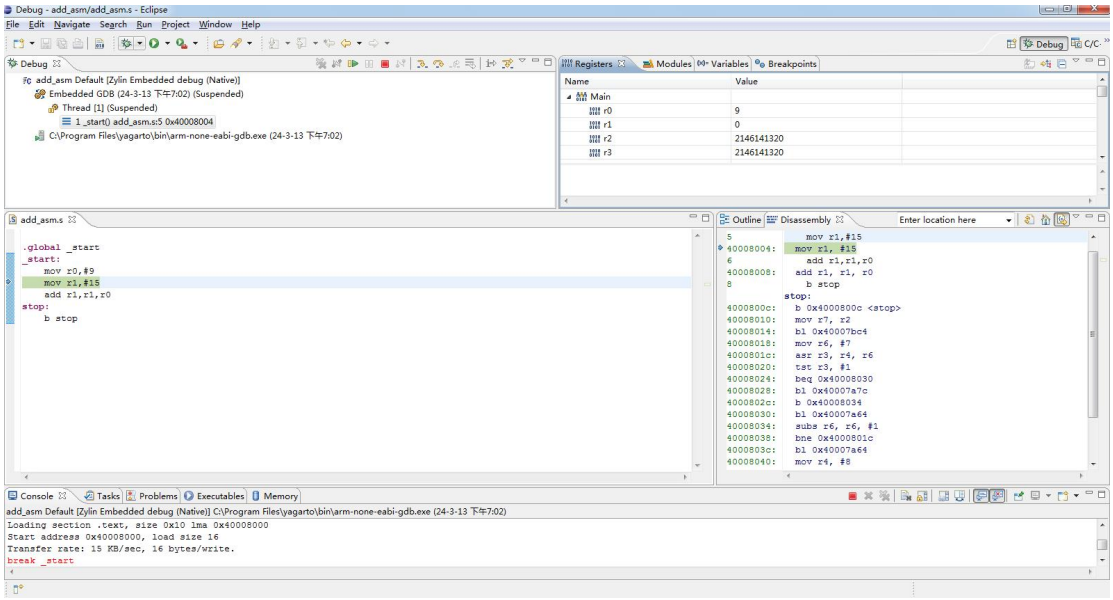
方法参考第 1 章 ARM 环境搭建。光盘实验源码路径：华清远见-CORTEXA9 资料\实验代码\1. ARM 体系结构与接口技术部分\实验代码\01-add_asm

2、打开“Register”显示框

单击 window -> show view -> Register，

3、单步仿真

配置完成之后，点击“  ”开始仿真，弹出 Debug 框。



单击“  ”单步仿真。查看仿真现象。

（2）伪代码实验

实验代码：

```
    .text
    .global _start
    _start:
            mov     r3,r3
            ldr     r0,=myarray
            mov     r4,#0
    loop:
            ldr     r1,[r0],#4
            add     r4,r4,r1
            cmp     r1,#0
            bne     loop
    stop:
            b       stop
    myarray:
            .word   0x11
            .word   0x22
            .word   0x0
            .end
```

实验步骤与（1）类似。

五、实验数据（现象）处理分析

（1）ADD 实验

① **初始状态：**R0 = 0，R1 = 0。

② **mov r0,#9：**将立即数 9 存入 R0 寄存器，此时 R0 = 9，R1 = 0。

```
.global _start
_start:
    mov r0,#42
    mov r1,#6
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	0
r2	2146141320
r3	2146141320

③ **mov r1,#15：**将立即数 15 存入 R1 寄存器，此时 R0 = 9，R1 = 15

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	15

④ **add r1,r1,r0：**将 R1 和 R0 的值相加，结果存入 R1，此时 R0 = 9，R1 = 24。

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	24

⑤ 无条件跳转至 stop 标签，程序结束。

```
.global _start
_start:
    mov r0,#9
    mov r1,#15
    add r1,r1,r0
stop:
    b stop
```

Name	Value
Main	
r0	9
r1	24

（2）伪指令实验

① **mov r3,r3**，将 r3 寄存器设置为自身的值，没有变化。

② **ldr r0,=myarray**：将数组 myarray 的地址加载到 r0 中。

```
.text
.global _start
_start:
        mov     r3,r3
        ldr     r0,=myarray
        mov     r4,#0
loop:
        ldr     r1,[r0],#4
        add     r4,r4,r1
```

Name	Value
Main	
r0	1073774624
r1	0
r2	2146141320
r3	327155712
r4	0

③ **mov r4,#0**：将 r4 寄存器清零。

```
.text
.global _start
_start:
        mov     r3,r3
        ldr     r0,=myarray
        mov     r4,#0
loop:
        ldr     r1,[r0],#4
        add     r4,r4,r1
        cmp     r1,#0
        bne     loop
stop:
        b       stop
```

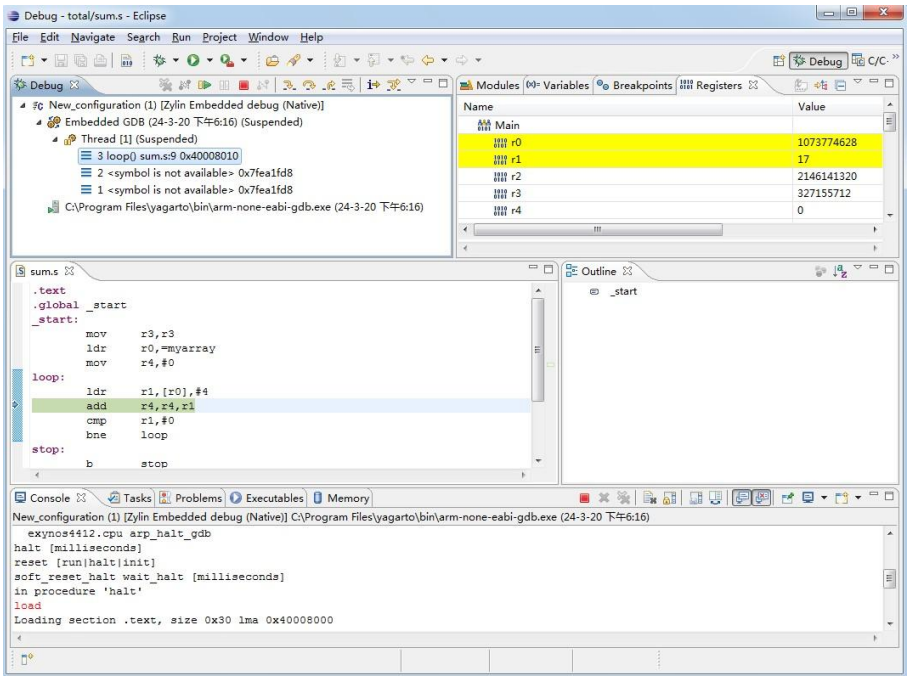
Modules | Variables | Breakpoints | Registers

Name	Value
Main	
r0	1073774624
r1	0
r2	2146141320
r3	327155712
r4	0

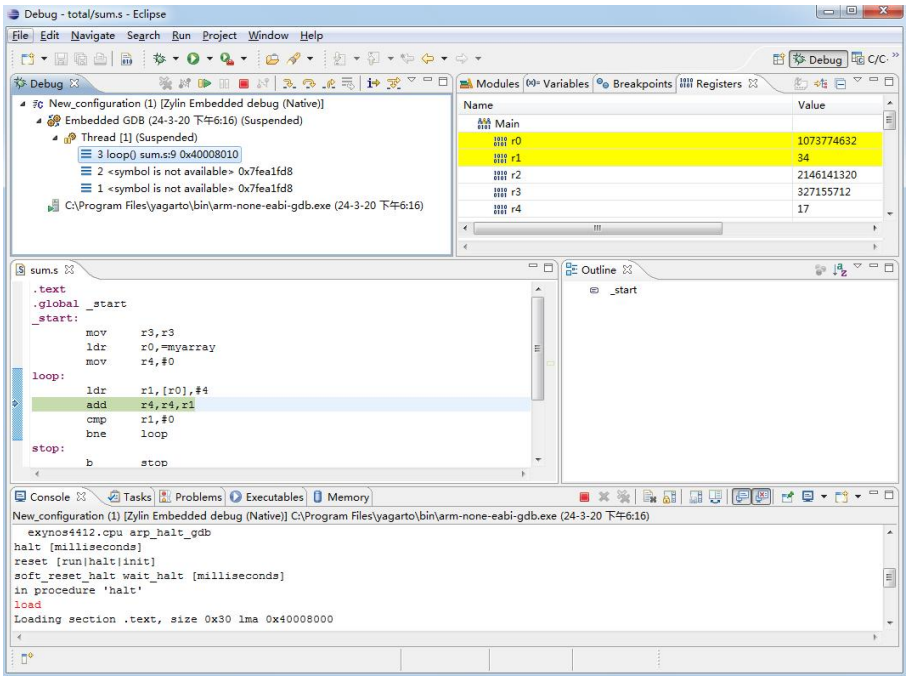
④ loop 标签处：

- **ldr r1,[r0],#4**：从 r0 指向的内存地址加载一个字（4 字节）到 r1 寄存器中，然后将 r0 的值增加 4。
- **add r4,r4,r1**：将 r4 寄存器中的值与 r1 寄存器中的值相加，结果存回 r4。
- **cmp r1,#0**：将 r1 寄存器的值与 0 进行比较，但不会改变寄存器的值。
- **bne loop**：如果上一次比较结果不等于，则跳转到 loop 标签处。否则继续执行下一条指令。

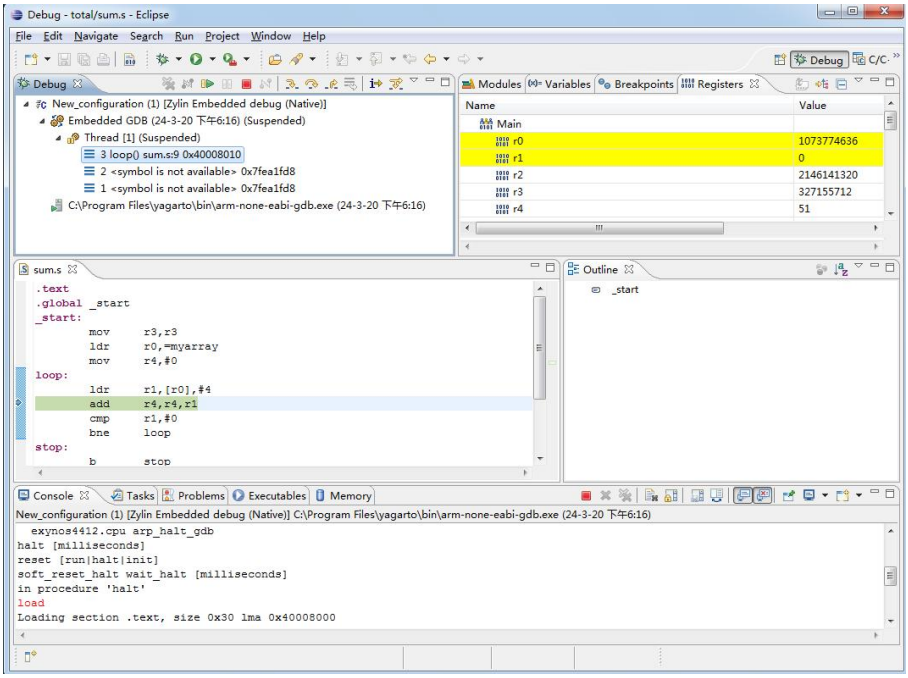
第一次循环：


Debug - total/sum.s - Eclipse
File Edit Navigate Search Run Project Window Help
Debug
New_configuration (1) [Zylin Embedded debug (Native)]
Embedded GDB (24-3-20 下午6:16) (Suspended)
Thread [1] (Suspended)
3 loop() sum.s:9 0x40008010
2 <symbol is not available> 0x7fea1fd8
1 <symbol is not available> 0x7fea1fd8
C:\Program Files\yagarto\bin\arm-none-eabi-gdb.exe (24-3-20 下午6:16)
Modules Variables Breakpoints Registers
Name Value
Main
r0 1073774628
r1 17
r2 2146141320
r3 327155712
r4 0
sum.s
.text
.global _start
_start:
mov r3,r3
ldr r0,=myarray
mov r4,#0
loop:
ldr r1,[r0],#4
add r4,r4,r1
cmp r1,#0
bne loop
stop:
b stop
Outline
_start
Console Tasks Problems Executables Memory
New_configuration (1) [Zylin Embedded debug (Native)] C:\Program Files\yagarto\bin\arm-none-eabi-gdb.exe (24-3-20 下午6:16)
exynos4412.cpu arp_halt_gdb
halt [milliseconds]
reset [run|halt|init]
soft_reset_halt wait_halt [milliseconds]
in procedure 'halt'
load
Loading section .text, size 0x30 lma 0x40008000

第二次循环：

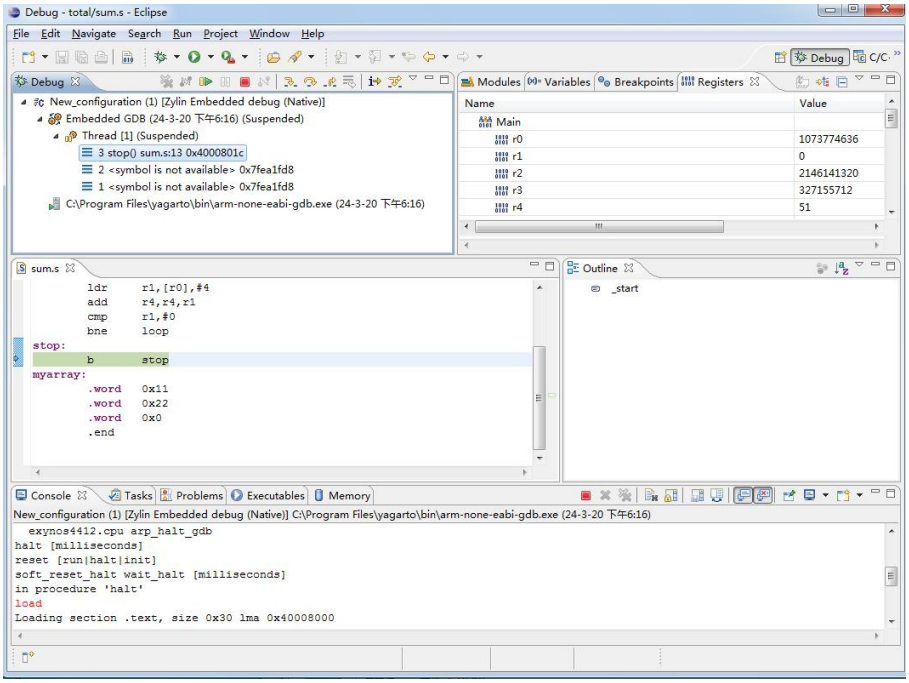

Debug - total/sum.s - Eclipse
File Edit Navigate Search Run Project Window Help
Debug
New_configuration (1) [Zylin Embedded debug (Native)]
Embedded GDB (24-3-20 下午6:16) (Suspended)
Thread [1] (Suspended)
3 loop() sum.s:9 0x40008010
2 <symbol is not available> 0x7fea1fd8
1 <symbol is not available> 0x7fea1fd8
C:\Program Files\yagarto\bin\arm-none-eabi-gdb.exe (24-3-20 下午6:16)
Modules Variables Breakpoints Registers
Name Value
Main
r0 1073774632
r1 34
r2 2146141320
r3 327155712
r4 17
sum.s
.text
.global _start
_start:
mov r3,r3
ldr r0,=myarray
mov r4,#0
loop:
ldr r1,[r0],#4
add r4,r4,r1
cmp r1,#0
bne loop
stop:
b stop
Outline
_start
Console Tasks Problems Executables Memory
New_configuration (1) [Zylin Embedded debug (Native)] C:\Program Files\yagarto\bin\arm-none-eabi-gdb.exe (24-3-20 下午6:16)
exynos4412.cpu arp_halt_gdb
halt [milliseconds]
reset [run|halt|init]
soft_reset_halt wait_halt [milliseconds]
in procedure 'halt'
load
Loading section .text, size 0x30 lma 0x40008000

第三次循环：



⑤ 当遇到数值为 0 的元素时，程序跳转到 stop 标签处，然后又跳回到自身，形成一个无限循环，最后 r1 结果为 0，r4 结果为 51。

⑥ stop 标签处，程序无限循环，不会改变任何寄存器的值。



六、实验结论

通过本次 ADD 实验和伪指令实验，我们深入学习了 ARM 汇编语言的基本使用以及一些常用的伪指令。通过编写和调试汇编代码，我们掌握了 mov、add、ldr、cmp 和 bne 等指令的语法和功能，并且熟悉了在 eclipse 开发工具下建立汇编工程和仿真的方法。通过实验数据的处理和分析，我们验证了实验代码的正确性，并且加深了对汇编语言程序执行过程的理解。这些实验为我们进一步学习和掌握 ARM 汇编语言奠定了基础。

练习题：

4、用 ARM 汇编实现下面列出的操作：

1) r0=15

MOV r0, #15

2) r0=r1/16（有符号数）

SXTB r0, r1 ; 将 r1 的低字节符号扩展到 r0

ASRS r0, r0, #4 ; 将 r0 右移 4 位，相当于除以 16

3) r1=r2*3

MUL R1, R2, #3

4) r0=-r0

RSBS r0, r0, #0

5、BIC 指令的作用是什么？

BIC 指令用于清除寄存器中的位

6、执行 SWI 指令时会发生什么？

执行 SWI 指令会触发软件中断，跳转到异常向量表中的 SWI 处理程序

7、B、BL、BX 指令的区别是什么？

B 指令:分支指令,跳转到标号指定的地址程序。

BL 指令:带链接的分支指令,跳转到指定的地址程序,同时还将程序计数器

PC 的值保存到 LR 寄存器中。

BX 指令:带状态切换的分支指令,跳转到 Rm 指定的地址处执行程序,根据跳转地址的最低位,来切换处理器状态。

8、下面哪个数据可以作为数据操作指令的有效立即数:

a．0x101　b．0x1f8　c．0xf000000f　d．0x08000012　e．0x104

b, c, e

9、ARM 在哪些工作工作模式下可以修改 CPSR 寄存器?

特权模式下才能修改 CPSR 寄存器, 即快速中断模式(FIQ)、外部中断模式(IRQ)、管理模式(SVC) 、数据访问中止模式(ABT)、未定义指令中止模式(UND)、系统模式(SYS)。

10、写一个程序，判断 R0 的值，大于 0x50，则将 R1 的值减去 0x10，并把结果送给 R0。

```
CMP r0, #0x50     ; 比较 r0 和 0x50
MOVGT r1, r1, #-0x10   ; 如果 r0 > 0x50，则将 r1 减去 0x10
MOVGT r0, r1      ; 将结果送给 r0
```

11、编写一段 ARM 汇编程序，实现数据块复制，将 R0 指向的 8 个字的连续数据保存到 R1 指向的一段连续的内存单元。

```
CopyLoop
    LDR r2, [r0], #4 ; 从 R0 指向的地址加载一个字，同时递增 R0 的值
    STR r2, [r1], #4 ; 将加载的字存储到 R1 指向的地址，同时递增 R1 的值
    SUBS r3, r3, #1   ; 计数器减 1
    BNE CopyLoop      ; 如果计数器不为 0，则继续复制
    END
```

1．在 GNU 风格的 ARM 汇编中如何定义一个全局的数字变量？

在 GNU 风格的 ARM 汇编中，要定义一个全局的数字变量，你可以使用 .globl 或 .global 指令来声明一个全局符号，然后使用 .word 指令为其分配空间并初始化。

2．AAPCS 中规定的 ARM 寄存器的使用规则是什么？

AAPCS（ARM 体系结构过程调用标准）规定了 ARM 寄存器的使用规则。根据 AAPCS，一些寄存器（如 r0 到 r3）用于函数参数传递和返回值。除此之外，寄存器 r4 到 r11 可以在函数内用作临时寄存器，但在函数调用之间必须进行保存。寄存器 r12 被称为 IP（Intra-Procedure-call）寄存器，可以由调用者或被调用者保存。寄存器 r13 是堆栈指针（SP），r14 是链接寄存器（LR），r15 是程序计数器（PC），它们有特殊的作用，应该由函数遵循 AAPCS 的规则使用。

3．什么是内联汇编？什么是嵌入型汇编？两者之间的区别是什么？

内联汇编：内联汇编允许直接在高级语言代码中插入汇编代码，通常使用特殊的语法来标记汇编代码的开始和结束。编译器会将内联汇编嵌入到生成的汇编代码中。在 C/C++中，通常使用 asm 关键字来指示内联汇编。

嵌入型汇编：嵌入型汇编是在高级语言代码中调用汇编函数或使用汇编文件。汇编代码通常作为单独的文件编写，然后通过链接器与高级语言代码一起编译。在 C/C++中，可以使用外部声明或使用汇编模块来定义嵌入型汇编。

主要区别在于内联汇编将汇编代码直接嵌入到高级语言代码中，而嵌入型汇编则是将汇编代码作为独立单元与高级语言代码分开。

4．汇编代码中如何调用 C 代码中定义的函数？

在汇编代码中调用 C 代码中定义的函数，你需要遵循适当的调用约定和链接规则。通常情况下，你需要在汇编代码中使用函数的符号名，并保证传递参数和处理返回值的正确性。在 GNU 工具链中，你可以使用 BL 指令（Branch with Link）调用 C 函数，并将返回值保存在适当的寄存器中。同时，你需要确保在链接时将 C 函数与汇编代码正确地链接起来。

七、教师批阅意见：   成绩评定：   教师签字： 年　　月　　日
八、备注：